Laporan Praktikum 4

Machine Learning

DISUSUN OLEH

Abdur Rohim
2411533015

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG
2026

A. PENDAHULUAN

Pembelajaran mesin merupakan salah satu cabang ilmu komputer yang berfokus pada pengembangan model yang mampu mempelajari pola dari data dan melakukan prediksi secara otomatis. Dalam proses pembentukan model, seringkali muncul permasalahan yang dikenal sebagai overfitting dan underfitting. Overfitting terjadi ketika model terlalu kompleks sehingga mampu menghafal data pelatihan, tetapi gagal dalam melakukan generalisasi pada data baru. Sebaliknya, underfitting terjadi ketika model terlalu sederhana sehingga tidak mampu menangkap pola yang ada dalam data secara optimal.

Untuk mengatasi permasalahan tersebut, diperlukan suatu teknik yang disebut regularization. Regularization merupakan metode yang digunakan untuk mengendalikan kompleksitas model dengan menambahkan penalti terhadap parameter model. Dengan adanya penalti ini, model diharapkan tidak terlalu bergantung pada data pelatihan dan mampu menghasilkan performa yang lebih baik pada data uji. Teknik ini menjadi sangat penting terutama dalam model regresi, di mana banyaknya fitur dapat memengaruhi stabilitas dan akurasi model.

Pada Praktikum kali ini kita akan mempelajari dua jenis regularization yang umum digunakan, yaitu Ridge Regression (L2) dan Lasso Regression (L1). Ridge berfungsi untuk mengecilkan nilai koefisien tanpa menghilangkannya, sedangkan Lasso dapat menghilangkan beberapa fitur dengan membuat koefisiennya menjadi nol. Melalui praktikum ini, mahasiswa diharapkan mampu memahami konsep, implementasi, serta dampak penggunaan regularization terhadap performa model pembelajaran mesin.

B. TUJUAN

1. Kita mampu memahami konsep dasar overfitting dan underfitting dalam pembelajaran mesin.
2. Kita mampu menjelaskan perbedaan antara metode Ridge Regression (L2) dan Lasso Regression (L1).
3. Kita mampu mengimplementasikan teknik regularization menggunakan library scikit-learn.

C. ISI

1. Pertama-tama yang harus kita lakukan adalah membuat kodingan yang berfungsi untuk mempersiapkan data yang akan kita kerjakan dan melakukan visualisasi terhadap instruksi yang kita berikan

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler

import warnings
warnings.filterwarnings("ignore")
```

2. Selanjutnya yang bisa kita lakukan adalah membuat kodingan yang berfungsi untuk menconnectkan google collabs dan google drive.

```
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive
```

3. Setelah itu kita akan membuat kodingan yang berfungsi untuk memanggil kembali dataset yang akan kita gunakan pada praktikum kali ini

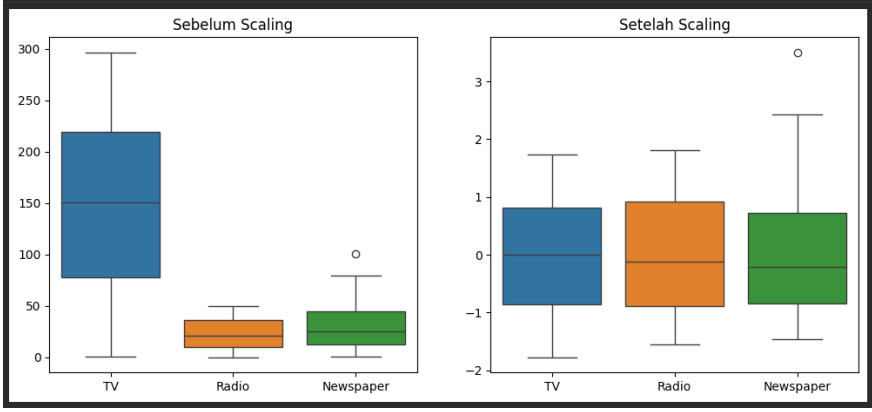
```
DATA_PATH = "/content/drive/MyDrive/Dataset/advertising.csv"
df = pd.read_csv(DATA_PATH)

X = df.drop("Sales", axis=1)
y = df["Sales"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

4. Kemudian kita buat kodingan yang akan berfungsi untuk menghilangkan bias akibat perbedaan skala dan menjaga validitas evaluasi model dengan tidak menggunakan informasi dari data yang sedang di proses.

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

5. Selanjutnya kita akan membuat kodingan yang berfungsi untuk membandingkan data sebelum dan sesudah scalling dan menampilkan gravik yang memberikan penjelasan bahwa scalling tidak mengubah pola distribusi secara signifikan.

```
fig, ax = plt.subplots(1,2, figsize=(12,5))
sns.boxplot(data=X_train[['TV','Radio','Newspaper']], ax=ax[0])
ax[0].set_title("Sebelum Scaling")

X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X_train.columns)
sns.boxplot(data=X_train_scaled_df[['TV','Radio','Newspaper']], ax=ax[1])
ax[1].set_title("Setelah Scaling")
plt.show()
```



6. Kemudian kita buat kodingan yang berfungsi untuk membuat dan melatih model serta melakukan prediksi terhadap data yang sudah kita latih

```
lr = LinearRegression()
lr.fit(X_train_scaled, y_train)

y_pred_train_lr = lr.predict(X_train_scaled)
y_pred_test_lr = lr.predict(X_test_scaled)

rmse_train_lr = np.sqrt(mean_squared_error(y_train, y_pred_train_lr))
rmse_test_lr = np.sqrt(mean_squared_error(y_test, y_pred_test_lr))

print("Linear Regression")
print("RMSE Train:", rmse_train_lr)
print("RMSE Test :", rmse_test_lr)
print("Coefficients:", lr.coef_)
```

```
Linear Regression
RMSE Train: 1.6358920055378559
RMSE Test : 1.7052146229349234
Coefficients: [4.58720774 1.48984025 0.08791597]
```

7. Setelah itu kita akan membuat kodingan yang akan berfungsi untuk membuat dan melatih model Ridge Regression dan memprediksi data yang sudah kita training dan di testing.

```
ridge = Ridge(alpha=1.0)
ridge.fit(X_train_scaled, y_train)

y_pred_train_ridge = ridge.predict(X_train_scaled)
y_pred_test_ridge = ridge.predict(X_test_scaled)

rmse_train_ridge = np.sqrt(mean_squared_error(y_train, y_pred_train_ridge))
rmse_test_ridge = np.sqrt(mean_squared_error(y_test, y_pred_test_ridge))

print("\nRidge Regression (alpha=1.0)")
print("RMSE Train:", rmse_train_ridge)
print("RMSE Test :", rmse_test_ridge)
print("Coefficients:", ridge.coef_)
```

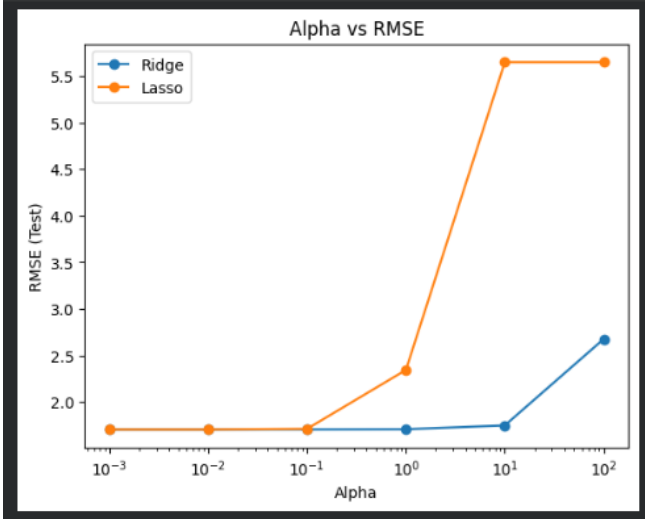
8. Kemudian kita perlu membuat kodingan yang berguna untuk membuat dan melatih model Lasso Regression dan memprediksi data yang sudah kita training dan di testing.

```
lasso = Lasso(alpha=0.1)
lasso.fit(X_train_scaled, y_train)

y_pred_train_lasso = lasso.predict(X_train_scaled)
y_pred_test_lasso = lasso.predict(X_test_scaled)

rmse_train_lasso = np.sqrt(mean_squared_error(y_train, y_pred_train_lasso))
rmse_test_lasso = np.sqrt(mean_squared_error(y_test, y_pred_test_lasso))

print("\nLasso Regression (alpha=0.1)")
print("RMSE Train:", rmse_train_lasso)
print("RMSE Test :", rmse_test_lasso)
print("Coefficients:", lasso.coef_)
```

9. Selanjutnya kita akan membuat kodingan yang berfungsi untuk Menganalisis pengaruh parameter alpha terhadap performa model dan membandingkan antara ridge dan lasso

regression dalam berbagai Tingkat dan memvissualisasikan hasil agar lebih mudah dintrepresentasikan.

```
alphas = [0.001, 0.01, 0.1, 1, 10, 100]

ridge_rmse = []
lasso_rmse = []

for a in alphas:
    r = Ridge(alpha=a).fit(X_train_scaled, y_train)
    l = Lasso(alpha=a).fit(X_train_scaled, y_train)

    ridge_rmse.append(np.sqrt(mean_squared_error(y_test, r.predict(X_test_scaled))))
    lasso_rmse.append(np.sqrt(mean_squared_error(y_test, l.predict(X_test_scaled))))

plt.plot(alphas, ridge_rmse, marker='o', label='Ridge')
plt.plot(alphas, lasso_rmse, marker='o', label='Lasso')
plt.xscale("log")
plt.xlabel("Alpha")
plt.ylabel("RMSE (Test)")
plt.legend()
plt.title("Alpha vs RMSE")
plt.show()
```



Latihan

1. Kerjakan dan Cobakan Latihan di bawah ini
   a. Apakah Ridge menurunkan gap RMSE antara train dan test?

```
gap_lr = abs(rmse_test_lr - rmse_train_lr)

ridge_gap_results = []

for a in alphas:
    ridge = Ridge(alpha=a)
    ridge.fit(X_train_scaled, y_train)

    y_pred_train_ridge = ridge.predict(X_train_scaled)
    y_pred_test_ridge = ridge.predict(X_test_scaled)

    rmse_train_ridge = np.sqrt(mean_squared_error(y_train, y_pred_train_ridge))
    rmse_test_ridge = np.sqrt(mean_squared_error(y_test, y_pred_test_ridge))
    gap_ridge = abs(rmse_test_ridge - rmse_train_ridge)

    ridge_gap_results.append({
        "Alpha": a,
        "RMSE Train Ridge": rmse_train_ridge,
        "RMSE Test Ridge": rmse_test_ridge,
        "Gap RMSE Ridge": gap_ridge
    })

ridge_gap_df = pd.DataFrame(ridge_gap_results)
```

```python
ridge_gap_df = pd.DataFrame(ridge_gap_results)

print("Gap RMSE Linear Regression")
print("RMSE Train Linear Regression:", rmse_train_lr)
print("RMSE Test Linear Regression :", rmse_test_lr)
print("Gap Linear Regression       :", gap_lr)

print("\nGap RMSE Ridge Regression")
display(ridge_gap_df)

ridge_gap_lebih_kecil = ridge_gap_df[ridge_gap_df["Gap RMSE Ridge"] < gap_lr]

print("\nJawaban Poin A:")

if len(ridge_gap_lebih_kecil) > 0:
    print("Ya, Ridge menurunkan gap RMSE pada beberapa nilai alpha.")
    print("Alpha yang menghasilkan gap lebih kecil dari Linear Regression:")
    display(ridge_gap_lebih_kecil)
else:
    print("Tidak, Ridge belum menurunkan gap RMSE dibandingkan Linear Regression.")
```

Dengan Output :

```
Gap RMSE Linear Regression
RMSE Train Linear Regression: 1.6358920055378559
RMSE Test Linear Regression : 1.7052146229349234
Gap Linear Regression       : 0.06932261739706758

Gap RMSE Ridge Regression
```

	Alpha	RMSE Train Ridge	RMSE Test Ridge	Gap RMSE Ridge
0	0.001	1.635892	1.705217	0.069325
1	0.010	1.635892	1.705234	0.069342
2	0.100	1.635895	1.705412	0.069517
3	1.000	1.636162	1.707430	0.071269
4	10.000	1.659899	1.748760	0.088861
5	100.000	2.466618	2.676301	0.209683

```
Jawaban Poin A:
Tidak, Ridge belum menurunkan gap RMSE dibandingkan Linear Regression.
```

b. Apakah Lasso membuat beberapa koefisien menjadi nol?

```python
lasso_coef_results = []

for a in alphas:
    lasso = Lasso(alpha=a, max_iter=10000)
    lasso.fit(X_train_scaled, y_train)

    jumlah_koefisien_nol = np.sum(np.isclose(lasso.coef_, 0, atol=1e-6))

    lasso_coef_results.append({
        "Alpha": a,
        "Koefisien TV": lasso.coef_[0],
        "Koefisien Radio": lasso.coef_[1],
        "Koefisien Newspaper": lasso.coef_[2],
        "Jumlah Koefisien Nol": jumlah_koefisien_nol
    })

lasso_coef_df = pd.DataFrame(lasso_coef_results)

print("Koefisien Lasso Regression")
display(lasso_coef_df)

lasso_koefisien_nol = lasso_coef_df[lasso_coef_df["Jumlah Koefisien Nol"] > 0]

print("\nJawaban Poin B:")

if len(lasso_koefisien_nol) > 0:
    print("Ya, Lasso membuat beberapa koefisien menjadi nol pada alpha tertentu.")
    print("Alpha yang menghasilkan koefisien nol:")
    display(lasso_koefisien_nol)
else:
    print("Tidak, pada nilai alpha yang diuji Lasso belum membuat koefisien menjadi nol.")
```

Dengan Output :

```
Koefisien Lasso Regression
```

	Alpha	Koefisien TV	Koefisien Radio	Koefisien Newspaper	Jumlah Koefisien Nol
0	0.001	4.586252	1.489189	0.087187	0
1	0.010	4.577701	1.483172	0.080685	0
2	0.100	4.492181	1.423016	0.015662	0
3	1.000	3.638166	0.575085	0.000000	1
4	10.000	0.000000	0.000000	0.000000	3
5	100.000	0.000000	0.000000	0.000000	3

```
Jawaban Poin B:
Ya, Lasso membuat beberapa koefisien menjadi nol pada alpha tertentu.
Alpha yang menghasilkan koefisien nol:
```

	Alpha	Koefisien TV	Koefisien Radio	Koefisien Newspaper	Jumlah Koefisien Nol
3	1.0	3.638166	0.575085	0.0	1
4	10.0	0.000000	0.000000	0.0	3
5	100.0	0.000000	0.000000	0.0	3

c. Pada nilai lambda berapa RMSE test paling kecil?

```python
all_results = []

all_results.append({
    "Model": "Linear Regression",
    "Alpha": 0,
    "RMSE Train": rmse_train_lr,
    "RMSE Test": rmse_test_lr
})

for a in alphas:
    ridge = Ridge(alpha=a)
    ridge.fit(X_train_scaled, y_train)

    y_pred_train_ridge = ridge.predict(X_train_scaled)
    y_pred_test_ridge = ridge.predict(X_test_scaled)

    all_results.append({
        "Model": "Ridge Regression",
        "Alpha": a,
        "RMSE Train": np.sqrt(mean_squared_error(y_train, y_pred_train_ridge)),
        "RMSE Test": np.sqrt(mean_squared_error(y_test, y_pred_test_ridge))
    })

for a in alphas:
    lasso = Lasso(alpha=a, max_iter=10000)
    lasso.fit(X_train_scaled, y_train)

    y_pred_train_lasso = lasso.predict(X_train_scaled)
    y_pred_test_lasso = lasso.predict(X_test_scaled)

    all_results.append({
        "Model": "Lasso Regression",
        "Alpha": a,
        "RMSE Train": np.sqrt(mean_squared_error(y_train, y_pred_train_lasso)),
        "RMSE Test": np.sqrt(mean_squared_error(y_test, y_pred_test_lasso))
    })

all_results_df = pd.DataFrame(all_results)

print("Semua Hasil RMSE")
display(all_results_df)

best_result = all_results_df.loc[all_results_df["RMSE Test"].idxmin()]

print("\nJawaban Poin C:")
print("RMSE test paling kecil diperoleh oleh:")
print("Model :", best_result["Model"])
print("Alpha :", best_result["Alpha"])
print("RMSE Test :", best_result["RMSE Test"])
```

Dengan Output :

```
Semua Hasil RMSE
```

	Model	Alpha	RMSE Train	RMSE Test
0	Linear Regression	0.000	1.635892	1.705215
1	Ridge Regression	0.001	1.635892	1.705217
2	Ridge Regression	0.010	1.635892	1.705234
3	Ridge Regression	0.100	1.635895	1.705412
4	Ridge Regression	1.000	1.636162	1.707430
5	Ridge Regression	10.000	1.659899	1.748760
6	Ridge Regression	100.000	2.466618	2.676301
7	Lasso Regression	0.001	1.635893	1.705184
8	Lasso Regression	0.010	1.635964	1.705030
9	Lasso Regression	0.100	1.643031	1.711768
10	Lasso Regression	1.000	2.140203	2.343866
11	Lasso Regression	10.000	5.176811	5.648217
12	Lasso Regression	100.000	5.176811	5.648217

```
Jawaban Poin C:
RMSE test paling kecil diperoleh oleh:
Model : Lasso Regression
Alpha : 0.01
RMSE Test : 1.7050297827636898
```

d. Apakah lambda terlalu besar menyebabkan underfitting?

```python
alpha_besar = max(alphas)

hasil_alpha_besar = all_results_df[
    (all_results_df["Alpha"] == alpha_besar) &
    (all_results_df["Model"].isin(["Ridge Regression", "Lasso Regression"]))
]

print("Hasil Model dengan Alpha Terbesar")
display(hasil_alpha_besar)

print("\n=== Pembanding Linear Regression ===")
print("RMSE Train Linear Regression:", rmse_train_lr)
print("RMSE Test Linear Regression :", rmse_test_lr)

print("\nJawaban Poin D:")

for index, row in hasil_alpha_besar.iterrows():
    if row["RMSE Train"] > rmse_train_lr and row["RMSE Test"] > rmse_test_lr:
        print(
            row["Model"],
            "dengan alpha",
            alpha_besar,
            "kemungkinan mengalami underfitting karena RMSE train dan RMSE test lebih besar dibanding Linear Regression."
        )
    else:
        print(
            row["Model"],
            "dengan alpha",
            alpha_besar,
            "belum menunjukkan underfitting yang kuat."
        )
```

Dengan Output :

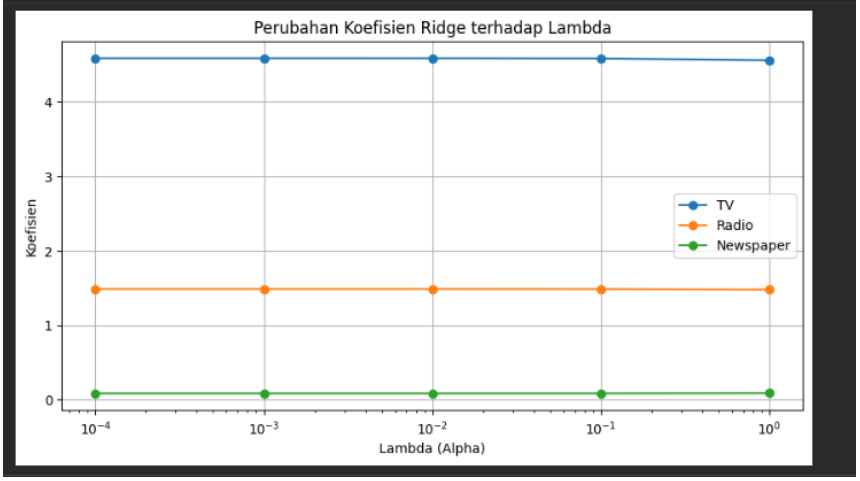
```
Hasil Model dengan Alpha Terbesar
```

	Model	Alpha	RMSE Train	RMSE Test
6	Ridge Regression	100.0	2.466618	2.676301
12	Lasso Regression	100.0	5.176811	5.648217

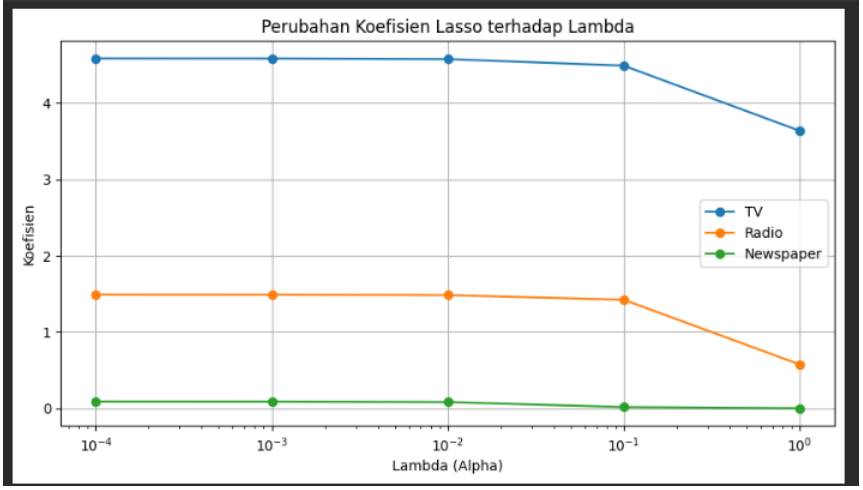
```
=== Pembanding Linear Regression ===
RMSE Train Linear Regression: 1.6358920055378559
RMSE Test Linear Regression : 1.7052146229349234

Jawaban Poin D:
Ridge Regression dengan alpha 100 kemungkinan mengalami underfitting karena RMSE train dan RMSE test lebih besar dibanding Linear Regression.
Lasso Regression dengan alpha 100 kemungkinan mengalami underfitting karena RMSE train dan RMSE test lebih besar dibanding Linear Regression.
```

2. Lakukan Percobaan Berikut ini
   a. Uji minimal dengan 5 nilai lambda berbeda selain nilai pada latihan praktikum dan buatkan grafik nya.

```python
plt.figure(figsize=(10,5))

for i in range(ridge_coefs.shape[1]):
    plt.plot(alphas, ridge_coefs[:, i], marker='o', label=X.columns[i])

plt.xscale('log')
plt.xlabel("Lambda (Alpha)")
plt.ylabel("Koefisien")
plt.title("Perubahan Koefisien Ridge terhadap Lambda")
plt.legend()
plt.grid()
plt.show()
```




```python
plt.figure(figsize=(10,5))

for i in range(lasso_coefs.shape[1]):
    plt.plot(alphas, lasso_coefs[:, i], marker='o', label=X.columns[i])

plt.xscale('log')
plt.xlabel("Lambda (Alpha)")
plt.ylabel("Koefisien")
plt.title("Perubahan Koefisien Lasso terhadap Lambda")
plt.legend()
plt.grid()
plt.show()
```



b. Jelaskan perbedaan perilaku/pengaruh Ridge vs Lasso dalam konteks dataset ini

Dalam konteks dataset ini, Ridge Regression cenderung mempertahankan seluruh variabel dengan mengurangi besar pengaruhnya secara proporsional, sehingga cocok untuk menjaga stabilitas model.

Sebaliknya, Lasso Regression tidak hanya mengurangi besar koefisien, tetapi juga mampu menghilangkan variabel yang kurang relevan, sehingga menghasilkan model yang lebih sederhana dan mudah diinterpretasikan.

D. KESIMPULAN

Pada praktikum kali ini dapat kita pahami bahwasnya regularization merupakan teknik yang efektif untuk mengatasi permasalahan overfitting dalam model pembelajaran mesin dengan cara menambahkan penalti pada fungsi loss. Penambahan penalti ini membuat model menjadi lebih sederhana sehingga mampu melakukan generalisasi dengan lebih baik terhadap data baru. Hal ini terlihat dari adanya peningkatan keseimbangan performa antara data pelatihan dan data pengujian setelah penerapan regularization.

Ridge Regression bekerja dengan mengecilkan nilai koefisien tanpa menghilangkannya sehingga seluruh fitur tetap digunakan, sedangkan Lasso Regression dapat menghilangkan fitur yang tidak relevan dengan membuat koefisiennya menjadi nol. Selain itu, parameter lambda memiliki peran penting dalam menentukan kinerja model, di mana nilai yang terlalu kecil dapat menyebabkan overfitting dan nilai yang terlalu besar dapat menyebabkan underfitting. Oleh karena itu, pemilihan nilai lambda yang tepat sangat diperlukan untuk menghasilkan model yang optimal.

Link Repo : https://github.com/Rohim110606/AbdulRohim_2411533015_ML2526.git